Coverage Guide

Functional Coverage Guide (SV-only SPI Master Verification)

0) What functional coverage is (quick learning section)

What is functional coverage?

Functional coverage answers:

> "Did our tests *exercise the important behaviors and corner cases* the spec cares about?"

Unlike code coverage (statement/branch), functional coverage is **intentional**: you define bins for key configuration/state/events, then you "sample" them when they occur.

Why it matters in your project

Your rubric requires:

- **Functional coverage ≥ 85%** (on golden RTL)
- You also want high bug-detection against 28 buggy DUT variants
  Coverage helps ensure you're not missing entire parts of configuration space.

Key idea

You need two things:

1. **Covergroups and bins** (what to measure)
2. **Correct sampling points** (when to measure)

Your current `spi_coverage_col` has the beginnings of (1), but you must expand it greatly and make sampling consistent across tests and monitors.

---

1) Inputs to use (from your scaffold)

Your `tb_top.sv` already creates a coverage object:

- `spi_coverage_col u_cov = new();`

and tests call:

- `coverage.sample_config(mode, lsb_first, width);`

This is good: it means the coverage collector is a **class** that tests can access via ref, and you can add new sample tasks safely.

Important constraint

Since you are **SV-only**, not UVM:

- You do not have UVM analysis ports.
- You will sample coverage by calling coverage tasks from:
  - tests (config intent)
  - BFMs/monitors (observed behavior)
  - scoreboard/ref_model (events detected)

This is totally valid.

---

2) Coverage requirements you must implement (from spec Section 10.1)

Minimum functional coverage targets listed in the spec:

1. **All 4 SPI modes × all 3 widths × MSB/LSB-first**
   - 4 × 3 × 2 = **24 bins**
2. **CLK_DIV corners**
   - 0,1,2,3,255,1024,65535 + random bin
3. **FIFO occupancy bins**
   - empty, 1, mid (4), 7, full for **TX and RX**
4. **Each of the 5 interrupts**
   - asserted
   - cleared via W1C
   - asserted while masked (status set, IRQ still 0)
5. **Each register**
   - written
   - read back
   - reset value observed
6. **DELAY bins**
   - 0, 1, and ≥ 128
7. **Loopback mode**
   - at least one transfer per width

Your grading interface also implies:

- Your required tests should naturally hit these bins if written properly.
- But you must build coverage so you can *see* if you missed something.

---

3) Strategy: split coverage into 4 logical covergroups

To keep implementation modular and readable, create coverage in **four categories**:

CG1 — Configuration coverage (what settings did we try?)

- mode, width, lsb_first, loopback
- DELAY buckets
- CLK_DIV buckets

This coverage can be sampled from tests, because tests know what they programmed.

CG2 — Transfer/behavior coverage (what transactions happened?)

- transfer completed with given config
- ensures configuration is not only “written” but also “used in a real transfer”

This is best sampled on **transfer-done event** (from ref_model/monitor).

CG3 — FIFO state coverage (what FIFO depths did we hit?)

- TX occupancy bins
- RX occupancy bins
- overflow events

Sample this when occupancy changes:

- after TX_DATA write
- after core pops TX word
- after transfer pushes RX
- after RX_DATA read

If you can’t observe internal pointers, sample via STATUS bits + predicted occupancy in model.

CG4 — Interrupt coverage (events + W1C + masked behavior)

Sample when:

- INT_STAT bit becomes 1
- W1C happens
- event happens while masked

This is easiest from the scoreboard/ref_model because it already tracks “events happened”.

---

4) Decide sampling sources (very important)

Coverage is useless if you sample at the wrong time. Use this rule:

Sample “config coverage” at programming time

When tests write CTRL/CLK_DIV/DELAY, they should call something like:

- `coverage.sample_config( ... )`
- `coverage.sample_clk_div(div)`
- `coverage.sample_delay(delay)`
- `coverage.sample_loopback(loopback)`

Sample “transfer coverage” at transfer completion

When a word transfer completes (you know because BUSY drops, or TRANSFER_DONE interrupt, or SPI monitor counts WIDTH bits), then sample:

- `coverage.sample_transfer_done(mode,width,lsb_first,loopback)`

This prevents “fake coverage” where you configured mode 3 but never actually transmitted.

Sample “FIFO/IRQ coverage” on events

- TX occupancy changes after TX push / TX pop
- RX occupancy changes after RX push / RX pop
- IRQ events sampled when they occur, not just at end

---

5) Step-by-step implementation plan for the coverage owner

Step 1 — Expand the existing config covergroup

Your current `cg_config` covers:

- mode bins
- lsb_first bins
- width bins
- cross mode×width

You must extend it to meet the spec requirement (24 bins):

- Add cross **mode × width × lsb_first** (this ensures all 24 combinations hit)

Also add:

- loopback coverpoint (0/1)
- (optional) cross loopback×width to ensure “loopback per width”

Outcome: You can answer: “Did we test all combinations?”

Step 2 — Add clock divider coverage

Create a dedicated covergroup `cg_clkdiv` with bins:

- div0, div1, div2, div3
- div255
- div1024
- div65535
- random_range (everything else)

Sampling rule: sample whenever `CLK_DIV` is programmed, and ideally also when a transfer actually uses it.

Outcome: You hit the DIV corner cases required by spec.

Step 3 — Add delay coverage

Create `cg_delay` with bins:

- delay0
- delay1
- delay_ge_128
- (optional) middle bucket (2–127) to help overall %

Sample on DELAY programming; optionally sample again on transfer boundaries.

Step 4 — Add FIFO occupancy coverage (TX and RX)

You must cover occupancy bins:

- 0 (empty)
- 1
- 4 (mid)
- 7
- 8 (full)

Implementation choices:

Option A (recommended for SV-only beginners): sample occupancy from the reference model

Your scoreboard/ref_model already needs to track:

- how many TX words were pushed minus popped
- how many RX words were pushed minus read

So coverage owner adds:

- `coverage.sample_tx_level(level)` whenever model updates tx_level
- `coverage.sample_rx_level(level)` whenever model updates rx_level

This is robust even if DUT status bits are buggy (you still see your stimulus space).

Option B: sample from STATUS bits only

Possible, but less precise: STATUS gives full/empty but not 1/4/7. Harder to meet bins.

Outcome: You get occupancy bins for both FIFOs.

Step 5 — Add interrupt coverage

There are 5 interrupts:

- TX_EMPTY
- RX_FULL

- TX_OVF
- RX_OVF
- TRANSFER_DONE

You need coverage bins for each:

1. asserted
2. cleared via W1C
3. asserted while masked (INT_EN=0 but INT_STAT sets)

This is easiest using ref_model/scoreboard events because it knows:

- when an event happened
- when W1C was written
- whether INT_EN bit was set at that time

So implement sampling tasks like:

- `coverage.sample_irq_event(bit_id, masked)`
- `coverage.sample_irq_w1c(bit_id)`
- `coverage.sample_irq_asserted(bit_id)`

Then in interrupt_test, call the sampling tasks at the moment you perform the action.

Outcome: You meet the interrupt bins required.

Step 6 — Register coverage (write, readback, reset)

Implement per-register bins for the 9 registers:

- CTRL, STATUS, TX_DATA, RX_DATA, CLK_DIV, SS_CTRL, INT_EN, INT_STAT, DELAY

Each needs bins:

- reset observed
- write performed (RW ones)
- read performed / readback checked

This is best sampled from APB BFM monitor:

- any time a transaction occurs, detect address and read/write

So add to coverage collector a method like:

- `coverage.sample_apb_access(addr, is_write)`
  and optionally:
- `coverage.sample_reg_reset_seen(regname)` called from `reg_access_test`

Outcome: the “each register covered” requirement is met.

Step 7 — Add “transfer actually happened” coverage

To avoid false closure (config sampled but no real shift), add:

- coverpoint: transfer_done occurred at least once for each mode/width/order
- cross that with loopback maybe

Sampling can happen when:

- your SPI monitor reconstructs one transfer
- or when ref_model expects a transfer done and observes RX push

Outcome: ensures stimulus isn’t only register writes.

---

6) How the coverage owner should coordinate with other team members

Coverage owner needs hooks from:

1. **Test writers**
   - must call `coverage.sample_config( ... )` after programming CTRL
   - must call `coverage.sample_clkdiv(div)` after writing CLK_DIV
   - must call `coverage.sample_delay(delay)` after writing DELAY
   - must call `coverage.sample_ss(ss_en, ss_val)` if you add SS coverage
2. **Ref model / scoreboard owner**
   - must expose FIFO levels and IRQ events so coverage can sample them
   - must notify on transfer completion
3. **APB BFM/monitor owner**
   - should provide callback or simply call coverage sampling on every APB access
   - or maintain a mailbox/queue of APB transactions that coverage reads

Recommended “lowest-friction” method (SV-only)

Add in `spi_ref_model` methods that **call coverage sampling** at natural points (FIFO updates, IRQ updates). This centralizes coverage and reduces repeated calls across tests.

---

7) Validation: how to know you're "closing coverage"

Step-by-step closure workflow

1. Run the full regression on golden RTL:
   - `make regress`
2. Run:
   - `make cov`
3. Read `coverage_report.txt` and confirm:
   - "Functional Coverage: XX%"
4. If functional coverage < 85%:
   - identify missing bins (your simulator's coverage viewer helps)
   - map missing bins to tests:
     - missing mode/width/order → update `mode_coverage_test`
     - missing DIV bins → update `clk_div_corner_test`
     - missing delay bins → update `delay_transfer_test`
     - missing FIFO occupancy bins → update `fifo_stress_test`
     - missing masked interrupt bins → update `interrupt_test`

Common beginner failure modes

- Sampling config but not doing a transfer → bins show "covered" but bugs not caught
- Not covering LSB-first or mode 3 → big holes
- Only using one width (8) → fails 24-bin cross
- Interrupt test sets INT_EN but never checks masked behavior
- FIFO stress never reaches "7" occupancy (only empty/full)

---

8) What to aim for beyond the minimum (to catch more bugs)

To maximize catching seeded bugs, add coverage for **transitions** and **races**, not just static values:

- EN toggling 1→0 flush case (R3 side-effect)
- W1C race sampling (R18)
- "write config during BUSY" attempted (illegal but reveals bugs)

- SS changes between transfers (SS mapping bugs)

These can be “bonus” coverpoints so you can tell you exercised them.

---

9) Deliverable checklist for the coverage teammate

By the end, the coverage teammate must deliver:

1. Updated `env/coverage.sv` with covergroups for:
   - config cross (mode×width×lsb_first) + loopback
   - clk_div corners
   - delay bins
   - fifo occupancy TX/RX bins (0/1/4/7/8)
   - interrupts (assert, w1c, masked)
   - register access (per register)
2. A list of **where sampling occurs**, documented in comments:
   - which tests call what
   - which model/monitor calls what
3. A “coverage closure log”:
   - last run functional %
   - any bins intentionally excluded (avoid excluding unless necessary)

Extra Functional Coverage Bins (for maximum bug detection + “coverage closure”)

A) Why “extra bins” matter (learning)

Minimum bins (mode×width×order, DIV corners, etc.) prove you visited the **configuration space**.
But seeded bugs are often in:

- **state transitions** (enable/disable, empty→non-empty, busy edges)
- **race conditions** (W1C race, simultaneous events)
- **boundary edges** (7→8 FIFO, 8→7 FIFO, last bit, first bit)
- **cross-feature interactions** (loopback + LSB-first + width=32 + delay>0)
- **software misuse handling** (writes during BUSY, SS changes mid-transfer)

So you add bins that confirm you exercised these *stress points*.

B) Extra bins: what to add (organized list)

B1) Transfer lifecycle bins (BUSY, start/end, back-to-back)

What to cover

- BUSY rising edge observed (start-of-transfer)
- BUSY falling edge observed (end-of-transfer)
- back-to-back transfers while BUSY stays high (or with DELAY inserted)
- “single transfer only” vs “burst of N transfers”

Suggested bins

- `busy_start_seen` (at least once)
- `busy_end_seen` (at least once)
- `burst_len_1`, `burst_len_2_to_4`, `burst_len_8_plus`
- `idle_gap_0` (no delay between words when DELAY=0)
- `idle_gap_nonzero` (DELAY>0 case)

Sampling points

- SPI monitor detects SS asserted + first SCLK edge ⇒ start
- monitor detects last sample edge ⇒ end
- scoreboard detects “transfer complete” event

Why it catches bugs

- off-by-one in BUSY timing
- transfer_done interrupt wrong
- delay logic broken
- state machine stuck BUSY

B2) Configuration sampled-at-start bins (R25 “write while BUSY”)

Even if behavior is “illegal,” your spec states **certain fields are sampled at transfer start**. This is a classic seeded-bug target.

Suggested bins

For each field: `attempted_mid_busy_write_<field>`

- MODE
- WIDTH
- LSB_FIRST
- CLK_DIV
- DELAY

And outcome bins (measured by monitor):

- `mid_busy_write_no_effect_current_transfer`
- `mid_busy_write_applies_next_transfer`

Sampling points

- Test writes register while STATUS.BUSY=1 ⇒ sample “attempted”
- Compare observed transfer config vs expected ⇒ sample “no_effect” or “applies_next”

Why it catches bugs

- DUT incorrectly re-samples mode/width mid-transfer
- divider changes mid-transfer causing glitchy SCLK

B3) Mode/edge correctness bins (CPOL/CPHA tricky cases)

Your base bins ensure you *set* mode; extra bins ensure you saw **actual edge behavior**.

Suggested bins

Per mode:

- `mode0_first_edge_sampled`
- `mode1_first_edge_sampled`
- `mode2_first_edge_sampled`
- `mode3_first_edge_sampled`

Also:

- `sclk_idle_correct_when_busy0` per mode (pairs with SVA but functional coverage proves stimulus hit it)

Sampling points

- SPI monitor sees first sample edge and tags mode in effect

Why it catches bugs

- wrong CPHA interpretation (sample/launch swapped)
- CPOL idle wrong between words

B4) Bit-order corner bins (patterns that expose shift bugs)

A lot of shift-order bugs only show up with patterns that “stress” endpoints.

Patterns (bins)

For each width, create bins for:

- “LSB only set” (e.g., 0x01, 0x0001, 0x0000_0001)
- “MSB only set” (0x80, 0x8000, 0x8000_0000)
- alternating (0xAA / 0x55; 0xA5A5; 0xA5A5_5A5A)
- all ones / all zeros

Then cross:

- `pattern_class × lsb_first`

Sampling points

- At transfer completion: sample “pattern class used” + order

Why it catches bugs

- reverse-bit bugs
- off-by-one (missing first/last bit)
- wrong shift direction

B5) FIFO transition bins (the “danger edges”)

The minimum occupancy bins are good, but seeded bugs often happen on transitions:

Suggested bins (TX and RX separately)

- `level_7_to_8` (becoming full)

- `level_8_to_7` (leaving full)
- `level_1_to_0` (becoming empty)
- `level_0_to_1` (leaving empty)

Also event bins:

- `tx_write_when_full` (overflow attempt)
- `rx_read_when_empty`
- `rx_overflow_on_push_when_full`

Sampling points

- Best from reference model (it knows exact levels)
- Also sample when STATUS flags change (FULL/EMPTY)

Why it catches bugs

- depth mis-sized (7 or 9 instead of 8)
- FULL/EMPTY flag wrong edge
- overflow not sticky

B6) Interrupt behavior bins (masking, W1C, multi-event)

Minimum bins cover basic assert/mask/clear/race, but add these:

Suggested bins per interrupt bit

- `irq_event_while_masked` (INT_EN=0 at event)
- `irq_event_while_enabled` (INT_EN=1 at event)
- `w1c_single_bit_clear`
- `w1c_write_0_no_effect`
- `w1c_clear_all_bits`
- `w1c_race_event_wins` (R18)
- `w1c_race_clear_wins` (if spec allows? usually not—so you should not expect this, but bin can track whether you tried the scenario)

Multi-interrupt interaction bins

- `multiple_int_bits_set_same_transfer`
  - e.g., TRANSFER_DONE + TX_EMPTY same time

- RX_FULL + RX_OVF sequence

Sampling points

- Scoreboard/ref_model is best: it knows events + masks + clear operations
- Sample on INT_STAT writes and on event detection

Why it catches bugs

- IRQ equation wrong (R16)
- INT_STAT not sticky
- W1C clears wrong bit or clears too much
- masking incorrectly blocks INT_STAT instead of just IRQ
- race handling incorrect

B7) SS behavior bins (mapping + stability)

Minimum SS bins check combinational behavior. Extra bins check real use patterns.

Suggested bins

- `ss_lane_used[0..3]`
- `ss_switch_between_transfers` (SS0 then SS1 etc.)
- `ss_asserted_during_transfer` (observed low during busy)
- `ss_deasserted_between_transfers` (if your tests do that)
- `ss_glitch_detected` (should be 0 hits; if it hits, it's a bug or test error)

Sampling points

- Direct observation of `spi.ss_n` during transfer (monitor)

Why it catches bugs

- wrong SS polarity or lane mapping
- SS not held stable due to internal logic mistake

B8) Loopback interaction bins (high bug value)

Loopback is a common place for muxing bugs.

Suggested bins

- `loopback_enabled_transfer_done`
- `loopback_enabled_width_8/16/32`
- `loopback_enabled_lsb_first` (if supported)
- `loopback_enabled_mode_3` (hard corner)

Sampling points

- On transfer completion, if loopback=1

Why it catches bugs

- loopback ignores width/order
- loopback still uses external MISO by mistake

B9) Delay interaction bins (DELAY + FIFO + mode)

Delay logic can break in subtle ways.

Suggested bins

- `delay0_back_to_back`
- `delay1_inserted`
- `delay_ge128_inserted`
- `delay_with_fifo_burst_len_8`
- `delay_with_mode3` (hard corner)

Sampling points

- Monitor measures idle half-cycles between transfers

Why it catches bugs

- delay off-by-one
- delay applied even when no queued word
- delay incorrectly drops BUSY

B10) APB access sequence bins (protocol + ordering)

Even if APB protocol is SVA-checked, coverage can confirm you actually exercised certain access types.

Suggested bins

- read-after-write same addr
- consecutive writes to same addr
- write CTRL while idle vs while BUSY
- reserved offset read/write performed
- TX_DATA write burst length bins (1, 2–4, 8+)
- RX_DATA read burst length bins

Sampling points

- APB BFM monitor captures every access: `(addr, is_write)`

Why it catches bugs

- register decode errors only show for certain sequences
- reserved offsets mishandled

C) How to implement extra bins safely (process guide)

Step C1 — Add bins only when you have a sampling source

Before adding a bin, answer:

- "Who will call `coverage.sample_*`?"
    - test? monitor? ref_model?
      If you can't identify a sampling point, do not add the bin yet.

Step C2 — Prefer "event-driven" sampling

Good: sample when transfer completes, interrupt asserted, FIFO level changes.
Bad: sample every cycle (too noisy, hard to debug).

Step C3 — Don't over-cross everything

Crossing too many coverpoints explodes bins and makes closure impossible.

Good crosses:

- mode × width × lsb_first (required)

- loopback × width (small)
- interrupt_id × masked (small)
- fifo_transition_type × fifo_kind (small)

Avoid:

- mode × width × lsb_first × div_bucket × delay_bucket × loopback
  (That will create hundreds/thousands of bins and you'll never close it.)

D) Practical "extra bins closure plan" (what tests should hit what)

Use the required tests as your main bin-hitters:

- `mode_coverage_test`
  - hits: mode×width×order, pattern classes, mode edge bins
- `clk_div_corner_test`
  - hits: div corners, mid-busy div write attempt bins
- `delay_transfer_test`
  - hits: delay0/1/≥128, idle-gap bins, delay-update-mid-busy
- `fifo_stress_test`
  - hits: fifo transitions 7↔8, 1↔0, burst length bins
- `interrupt_test`
  - hits: masked/unmasked, W1C, race bins, multi-event bins
- `loopback_test`
  - hits: loopback×width, loopback+mode corner (if you add it)
- `error_injection_test`
  - hits: tx overflow attempt, rx empty read, reserved offsets, illegal config attempt

---

E) "Extra bins" checklist for the coverage teammate

By the end, ensure you have:

1. **Transfer lifecycle bins** (start/end, burst lengths)
2. **Mid-busy write attempt bins** + effect bins (R25 stress)
3. **FIFO transition bins** (7→8, 8→7, 1→0, 0→1)
4. **Interrupt multi-event + masked + W1C race bins**

5. **Pattern class bins** cross bit order
6. **Loopback interaction bins** (width + order if possible)
7. **Delay interaction bins** (with burst and mode corners)
8. **SS lane usage and stability bins**
9. **APB sequence bins** (reserved accesses, read-after-write)

If you want, I can turn this into a **one-page "Coverage Owner Task Sheet"** (a checklist with priorities P0/P1/P2 and which test is responsible for each bin), so the person implementing coverage knows exactly what to do first to reach 85% quickly and then push toward maximum bug detection.